嵌入向量

Embeddings

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Embeddings	嵌入向量	
Token	词元	
Embedding	嵌入	
Representation	表示	
Hidden State	隐藏状态	
Transformer	Transformer 模型	Transformer
Retrieval	检索	
Vector	向量	
Matrix	矩阵	
Pooling	池化	
Model	模型	
Euclidean Distance	欧氏距离	
Cosine Similarity	余弦相似度	
Normalization	归一化	
Dot Product	点积	
Contrastive Learning	对比学习	
Queries	查询	
Models	模型	
Tokenization	分词	
Attention	注意力机制	Attention
Softmax	Softmax 归一化	Softmax
Reranking	重排序	
Chunking	分块	
Vocabulary	词表	
Training	训练	
Query	查询	Q
Mental Model	理解框架	

英文原词	中文译名	缩写／符号
Tokens	词元	
Embedding Matrix	嵌入矩阵	
Tokenizer	分词器	
Tensor	张量	
Matrix Multiplication	矩阵乘法	
Token Embedding	词元嵌入	
Allocator Overhead	分配器开销	
Precision	精度	
Weight Tying	权重绑定	
Gradient	梯度	
Pretraining	预训练	
Feed-forward	前馈	
Unembedding	解嵌入	
Hidden Representation	隐藏表示	
Contextual Embedding	上下文嵌入	
Activation	激活	
Document Embedding	文档嵌入	
RoPE	旋转位置嵌入	RoPE
Scaling	缩放	
Output Projection	输出投影	
Word Embedding	词嵌入	
Serving	推理服务	
Mask	掩码	
Mean Pooling	平均池化	
Decoder	解码器	
Sentence Embedding	句子嵌入	
Encoder	编码器	
Representation Space	表示空间	
Generation	生成	

英文原词	中文译名	缩写／符号
Benchmark	基准测试	
Baseline	基线	
Forward Pass	前向传播	
Dropout	随机失活	
Score Matrix	分数矩阵	
Retriever	检索器	
Index	索引	
Pipeline	流水线	
Quantization	量化	
Evidence	证据	
Accuracy	准确性	
Anisotropy	各向异性	
Embedding Space	嵌入空间	
Dynamic Range	动态范围	
Fine-tuning	微调	
Dense Vector	稠密向量	
Values	值	
Core	核心	
Recall	召回率	
Generator	生成器	
Reranker	重排序器	
Dtype	数据类型	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

嵌入向量 · 第 7 页

阅读导引 · 第 8 页

1 - 什么是嵌入（Embedding） · 第 10 页

2 - 为何采用分布式表示 · 第 11 页

3 - 词元（Token）嵌入矩阵（Embedding Matrix） · 第 12 页

4 - 查找是一个矩阵（Matrix）运算 · 第 13 页

5 - 参数计数和内存 · 第 14 页

6 - 如何学习嵌入向量（Embeddings） · 第 15 页

7 - 静态词嵌入向量（Embeddings） · 第 16 页

8 - 为什么静态字向量还不够 · 第 17 页

9 - 上下文嵌入向量（Embeddings） · 第 18 页

10 - 词元（Token）嵌入向量（Embeddings）与隐藏状态 · 第 19 页

11 - 位置和输入表示 · 第 20 页

12 - 输出嵌入向量（Embeddings）和权重绑定（Weight Tying） · 第 21 页

13 - 从词元（Tokens）到一个文本向量（Vector） · 第 22 页

14 - 池化（Pooling） · 第 23 页

15 - 原始 LM 状态并不是自动良好的语义嵌入向量（Embeddings） · 第 24 页

16 - 连体和双编码器（Encoder）模型（Models） · 第 25 页

17 - 相似性：点积（Dot Product）、余弦和 L2 · 第 26 页

18 - 标准化 · 第 27 页

19 - 工作几何形状示例 · 第 28 页

20 - 对比学习（Contrastive Learning） · 第 29 页

21 - 工作对比损失 · 第 30 页

22 - 负片和硬负片 · 第 31 页

23 - 稠密检索（Retrieval） · 第 32 页

24 - 查询（Query）/文档不对称和说明 · 第 33 页

25 - 嵌入（Embedding）尺寸 · 第 34 页

26 - 俄罗斯套娃代表 · 第 35 页

27 - 各向异性（Anisotropy）和表示（Representation）几何 · 第 36 页

28 - 单向量（Vector）、多向量和稀疏表示 · 第 37 页

29 - 多语言和多模式嵌入向量（Embeddings） · 第 38 页

30 - 评估嵌入向量（Embeddings） · 第 39 页

31 - RAG 中的嵌入向量（Embeddings） · 第 40 页

32 - 生产耦合和版本控制 · 第 41 页

33 - 透明的 Pytorch 实现 · 第 42 页
34 - 常见的实施错误 · 第 43 页
35 - 常见的误解 · 第 44 页
36 - 完整的理解框架（Mental Model） · 第 45 页
参考文献 · 第 46 页

嵌入向量

对应原稿第 1 页

理解

嵌入向量（**Embeddings**）

从词元（**Token**）查找表到现代语义表示

面向人工智能工程师、开发人员和认真学习者的简短技术手册

@techNmak

手册08 - 理解AI系列

阅读导引

对应原稿第 2 页

本手册将澄清哪些问题

“嵌入（Embedding）”是现代人工智能中最常用的词之一。它可能意味着学习中的一行

词元（Token）表、Transformer 内的上下文隐藏状态（Hidden State）、池化句子向量（Vector）、用于检索（Retrieval）的文档表示（Representation），甚至是跨文本和图像共享的向量。

本手册将这些含义分开并将它们联系起来。到最后，您应该能够解释：

• 为什么词元（Token）查找表是学习参数矩阵（Matrix）而不是定义数据库？

• 静态词向量与上下文词元（Token）状态有何不同；

• 可变长度文本如何通过池化（Pooling）或特定于模型（Model）的读出变成固定大小的向量（Vector）；

• 为什么余弦相似度（Cosine Similarity）、点积（Dot Product）、归一化（Normalization）和欧几里得距离不是可以互换的选择？

• 对比学习（Contrastive Learning）如何塑造空间，使有用的邻居变得亲密，而消极的邻居则变得可分离；

• 为什么检索（Retrieval）模型（Models）经常以不同的方式对待查询（Queries）和文档？

• 嵌入（Embedding）维数发生什么变化，以及何时截断在数学上是合理的；

• 为什么向量（Vector）空间只有与创建它的模型（Model）、预处理、目标和相似函数一起才有意义。

先备知识

对向量、矩阵、点积、softmax 和 Transformer 隐藏状态有基本的了解就足够了。早期关于标记化和注意力机制（Attention）的手册是有用的上下文，但是本手册从第一原理重建了嵌入（Embedding）机制。

适用范围

这是一本关于学习表征的手册。它涉及稠密检索（Retrieval）、RAG、多模态模型（Models）和向量（Vector）索引，只是为了解释如何生成和比较嵌入向量（Embeddings）。 HNSW、混合搜索、重排序（Reranking）、分块（Chunking）和完整检索系统设计都属于他们自己的手册。

全文使用的符号

V = 词汇量； T = 序列长度； d_model = Transformer 隐藏/词元（Token）嵌入（Embedding）宽度； d = 通用表示（Representation）宽度； d_e = 最终文本嵌入尺寸（当它与模型（Model）隐藏宽度不同时）。节定义了它们出现的任何附加局部符号。

研究依据：[1]、[4]、[7]、[9]、[10]、[11]、[14]、[18]

1 - 什么是嵌入（Embedding）

对应原稿第 3 页

向量（Vector）空间的学习映射

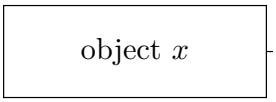
嵌入（Embedding）是将对象映射到有限维向量（Vector）的函数：

$$f(x) \in \mathbb{R}^d.$$

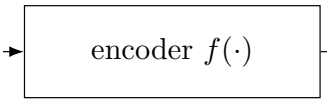
对象 x可能是一个词元（Token）ID、一个单词、一个句子、一个文档、一个图像、一个代码片段或其他完全不同的东西。有用的部分不在于输出是“数字列表”。有用的部分是训练（Training）使向量（Vector）空间中的关系支持下游目标。

对于词汇词元（Token），映射可能就像从矩阵（Matrix）中选择学习的行一样简单。对于一个句子，可能需要一个 Transformer 加上一个池化（Pooling）或读出规则。对于检索（Retrieval），训练（Training）目标可以明确鼓励相关查询（Query）-文档对具有高相似性。

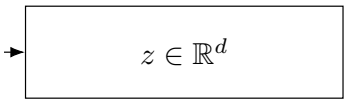
因此，嵌入（Embedding）不具有独立于其训练（Training）过程的通用语义意义。坐标417不会自动成为“否定”，坐标901也不会自动成为“主题”。表示（Representation）是分布式的：有用的信息通常跨多个维度和方向进行编码。



对象 x



编码器 $f(\cdot)$



理解框架（Mental Model）。嵌入（Embedding）是一种学习的表示（Representation），而不是提前传递给模型（Model）的语义坐标系。

研究依据：[1]、[2]、[3]、[13]

2 - 为何采用分布式表示

对应原稿第 4 页

通过共享统计强度进行概括

one-hot 表示（Representation）为每个词汇项提供了自己的独立轴。如果词汇表有 V条目，则 token i可以由 $\mathbb{R}^V$中的向缩放因子示，该向量（Vector）除了位置之外到处都为零

$$i.$$

该编码唯一地标识该词元（Tokens），但它不会使两个相关词元接近。“cat”和“dog”就像“cat”和“compiler”一样正交。相反，学习的密集表示（Representation）在更小的空间中为每个项目提供一个向量（Vector）$\mathbb{R}^d$,，在许多模型（Models）中为 $d \ll V$。

本吉奥等人。框架学习的连续单词表示作为对抗语言建模中维数灾难的一种方法：训练（Training）示例可以影响对附近表示的预测，而不仅仅是精确的离散序列。后来的词向量（Vector）方法，例如 word2vec 和 GloVe，使得大规模学习有用的几何结构变得更加便宜。 [1–3]

分发这个词很重要。通常不会为概念分配一个专用维度。每个向量（Vector）使用多个坐标，每个坐标可以参与表示多个项目。

重要。密集并不自动意味着语义。随机密集表仍然只是随机表。几何形状变得有用，因为目标和数据在训练（Training）过程中塑造了它。

研究依据：[1]、[2]、[3]

3 - 词元（Token）嵌入矩阵（Embedding Matrix）

对应原稿第 5 页

LLM 看到的第一个学习表示（Representation）

让词元（Token）生成器在 $\{0,\ldots,V-1\}$.中生成词元ID 标准词元嵌入（Embedding）层存储矩阵（Matrix）

$$E \in \mathbb{R}^{V \times d_{\text{model}}}.$$

Row E_i是与词元（Token） ID i.关联的学习向量（Vector）

对于词元（Token） ID 序列

$$(t_1, t_2, \ldots, t_n),$$

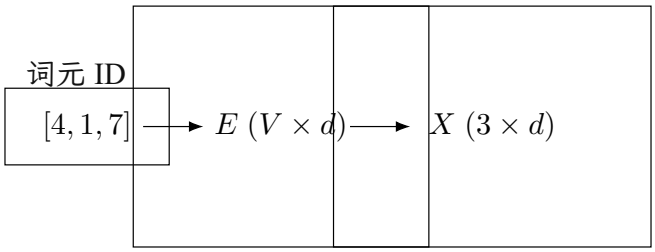
嵌入（Embedding）查找返回

$$X = [E_{t_1}; E_{t_2}; \ldots; E_{t_n}] \in \mathbb{R}^{n \times d_{\text{model}}}.$$

在批处理中，实现通常返回具有逻辑形状的张量（Tensor）

$$B \times T \times d_{\text{model}}.$$

PyTorch 将 nn.Embedding 直接描述为一个查找表，用于存储固定字典和嵌入向量（Embeddings）大小的嵌入向量。 [19]



嵌入（Embedding）行是模型（Model）参数。它不是词元（Token）生成器对词元的定义，并且词元 ID 本身不包含几何图形。 ID 仅选择行。

理解框架（Mental Model）。标记化选择离散符号。嵌入矩阵（Embedding Matrix）将这些符号转换为学习的连续向量。

研究依据：[4]、[7]、[19]

4 - 查找是一个矩阵（Matrix）运算

对应原稿第 6 页

One-hot 乘法和行选择是等价的

嵌入（Embedding）查找通常被实现为索引，因为它很高效，但它可以理解为普通的矩阵乘法（Matrix Multiplication）。

假设

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

1 0 1 0 2 1 3 1 0 2 2 2

$$E = \begin{bmatrix} 0 & 2 & 1 \\ 3 & 1 & 0 \\ 2 & 2 & 2 \end{bmatrix}$$

包含四个维度三的词元（Token）嵌入向量（Embeddings）。词元 ID 2 对应于 one-hot 行向量（Vector）

$$e_2^\top = [0, 0, 1, 0].$$

然后

$$e_2^\top E = [3, 1, 0],$$

这正是从零开始索引下的 E 的第 2 行。

对于序列，将 one-hot 行堆叠到 $O \in \mathbb{R}^{T \times V}$ 中。然后

$$X = OE \in \mathbb{R}^{T \times d}.$$

实际的实现通常不会构造巨大的稀疏独热矩阵（Matrix）。它仅收集请求的行。

这种等价性很有用，因为它消除了“嵌入（Embedding）层”的一些神秘感。它是从离散基础到低维连续空间的学习线性映射，通过索引进行有效评估。

重要。查找不会平均、搜索或检索最近的词元（Token）。给定一个 ID，该层会准确选择相应的参数行。

研究依据：[1]、[19]

5 - 参数计数和内存

对应原稿第 7 页

词汇量大小乘以嵌入（**Embedding**）宽度

词元嵌入（Token Embedding）表包含

$$V\, d_{\text{model}}$$

参数。这可能是较小语言模型（Model）的一个重要部分，尤其是当词汇量很大时。

举一个说明性的配置：

$$V = 32{,}768, \qquad d_{\text{model}} = 4096.$$

然后

$$Vd = 134{,}217{,}728$$

嵌入（Embedding）参数。每个参数两个字节，原始参数有效载荷为

$$268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

如果模型（Model）还使用相同形状的单独未绑定输出矩阵（Matrix），则在此示例中，该矩阵将需要另外 256 MiB 的两字节参数存储。

此计算仅是参数有效载荷。训练（Training）内存还可以包括梯度、优化器状态、更高精度（Precision）的主权重、分片元数据和分配器开销（Allocator Overhead）。

因此，词汇设计与模型（Model）大小相互作用。添加词元（Tokens）不是免费的：每个新词汇表行都会将 d_{model}学习参数添加到未绑定的输入表中。

重要。嵌入（Embedding）维度和词汇量大小是不同的轴。增加 V会添加行。增加 d会使每行变宽，如果词元嵌入（Token Embedding）直接为模型（Model）提供数据，还会更改隐藏宽度。

研究依据：[4]、[5]、[19]

6 - 如何学习嵌入向量（Embeddings）

对应原稿第 8 页

行移动是因为下游损失将梯度发回

嵌入（Embedding）表不需要特殊的“语义”损失即可变得有用。如果它参与可微模型（Model），任务损失可以通过反向传播更新所选行。

假设 词元（Token） ID t选择 row E_t.网络的其余部分计算损失 $\mathcal{L}$.反向传播提供

$$\frac{\partial \mathcal{L}}{\partial E_t}.$$

然后优化器会更新该行以及模型（Model）的其他参数。当前批次中未使用的词元（Tokens）的行可能不会在简单的嵌入（Embedding）层中接收直接查找梯度（Gradient），尽管权重绑定（Weight Tying）或矩阵（Matrix）的其他用途可以创建额外的梯度路径。

在许多示例中，参与类似预测上下文的词元（Tokens）可以被推向支持类似下游计算的表示。但并不要求欧几里得邻域成为人类可解释的，除非目标使几何变得有用。

早期的神经语言模型（Models）与概率模型联合显式学习分布式词向量。现代 Transformer 在更大的规模上做了同样的事情：词元（Token）嵌入向量（Embeddings）是与网络的其余部分端到端学习的。 [1,4]

理解框架（Mental Model）。嵌入（Embedding）表不是粘贴到模型（Model）上的预训练（Training）知识。在端到端模型中，它是由相同训练信号塑造的众多参数中的一组。

研究依据：[1]、[4]、[5]

7 - 静态词嵌入向量（Embeddings）

对应原稿第 9 页

Word2vec 和 GloVe 使矢量几何变得实用

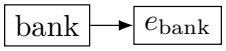
在上下文 Transformer 出现之前，一种常见的表示（Representation）形式为每种单词类型分配一个向量（Vector）。 Word2vec 引入了高效的预测架构，用于学习连续词向量

本地上下文，包括 CBOW 和skip-gram。 GloVe 相反，根据词与词共现统计数据训练全局对数双线性回归模型（Model）。 [2,3]

细节有所不同，但两个家族都帮助建立了一个持久的想法：文本中的分布规律可以在学习的向量（Vector）空间中编码为有用的几何形状。

静态嵌入向量（Embeddings）使得最近邻搜索、相似性、聚类和特征转移等操作变得廉价。他们还揭示了向量之间差异的惊人规律，尽管类比行为不应被误认为是意义的通用代数。

关键的限制在于“静态”一词。无论句子涉及金钱还是河流，诸如“存储体”之类的类型都会接收相同的基向量（Vector）。上下文只能稍后由另一个模型（Model）合并。



存储体

$\overline{e_{\text{bank}}}$

河岸同类型矢量

重要。一些词向量（Vector）空间著名的线性规律是学习表示的经验属性，并不能保证每个语义关系都是干净的向量偏移。

研究依据：[2]、[3]

8 - 为什么静态字向量还不够

对应原稿第 10 页

含义随上下文而变化

自然语言充满了一词多义和上下文依赖。每个单词类型的单个向量（Vector）必须将该类型的每次使用压缩为一种表示（Representation）形式。

上下文表示（Representation）模型（Models）改变了表示单位。 ELMo 使用双向语言模型状态生成单词表示作为整个输入句子的函数。然后，BERT 在预训练（Pretraining）期间通过在左右上下文中调节每个词元（Token）来学习深度双向 Transformer 表示。 [6,7]

现在“bank”出现两次

• “她在存储体存了现金。”

• “他们坐在河岸上。”

可以从相同的词元嵌入（Token Embedding）行开始，但在模型（Model）处理其周围环境后以不同的上下文隐藏状态结束。

这种区别是现代嵌入（Embedding）术语的基础：

静态词元（Token）/类型嵌入（Embedding） 与离散词汇项关联的学习向量（Vector）。上下文词元表示（Representation） 上下文计算后为词元出现而生成的隐藏状态（Hidden State）。

理解框架（Mental Model）。词元（Token）表提供了一个起始表示（Representation）。 Transformer 将该起点转变为依赖于上下文的状态。

研究依据：[6]、[7]

9 - 上下文嵌入向量（Embeddings）

对应原稿第 11 页

相同的 词元（Token） ID 可以产生不同的隐藏向量

让 token t_i具有基础嵌入 e_{t_i}·Transformer 层堆栈计算上下文状态

$$(h_1, \ldots, h_T) = F(e_{t_1}, \ldots, e_{t_T}; \theta).$$

尽管相同的 词元（Token） ID 无论出现在何处都会选择相同的基行 e_t，但 h_i取决于周围的 词元、位置处理、注意力机制（Attention）、前馈（Feed-forward）计算、归一化（Normalization）和正在检查的层。

因此，在没有指定具体表示（Representation）形式的情况下说“词元（Token） X 的嵌入（Embedding）”是不准确的。可能的含义包括：

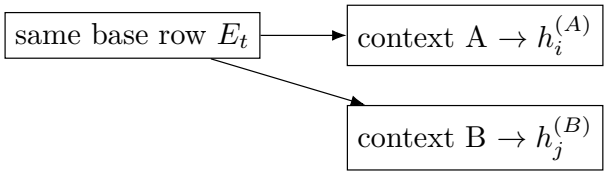
• 输入查找向量 E_t;

• ℓ;层一次出现的隐藏状态（Hidden State）

• 从许多词元（Token）状态派生的池化文本嵌入（Embedding）；

• 用于对词汇项进行评分的输出/非嵌入行。

上下文表示也是特定于层的。最终隐藏状态（Hidden State）不能与中间层互换，因为两者具有相同的维度。



相同基行 E_t

上下文A $\rightarrow h_i^{(A)}$

上下文B $\rightarrow h_i^{(B)}$

重要。“上下文嵌入（Contextual Embedding）”是通用语言，但对象通常是上下文生成器生成的隐藏表示（Hidden Representation），而不是存储在输入嵌入（Embedding）表中的行。

研究依据：[6]、[7]

10 - 词元（Token）嵌入向量（Embeddings）与隐藏状态

对应原稿第 12 页

不要折叠三种不同的向量（Vector）类型

现代语言模型（Models）公开了多个具有相同宽度的向量，这使得它们很容易混淆。

	词元嵌入	上下文状态	文本嵌入
Input	词元ID	序列状态是	text / token states
Context dependent?	no		usually yes
One per	vocab item	词元出现内部计算	sentence/document/etc.
Typical role	model input		similarity/retrieval/task feature

输入上下文相关？每个典型角色一个

词汇项模型（Model）输入

文本/词元（Token）状态通常是

句子/文件/等等。相似性/检索（Retrieval）/任务特征

第一个是参数查找。第二个是激活（Activation）。第三种是为了比较句子或文档等较大单位而故意公开的表示（Representation）。

将所有三种称为“嵌入向量（Embeddings）”不一定是错误的，但在推理记忆、训练（Training）、相似性或重用时会变得危险。词汇嵌入矩阵（Embedding Matrix）存在于模型（Model）参数中。上下文状态是根据输入生成的。文档嵌入（Document Embedding）通常可以预先计算并存储在模型外部。

理解框架（Mental Model）。总是问：嵌入的是什么，通过哪个功能，在哪一层或读出，以及出于什么目的？

研究依据：[4]、[7]、[9]、[10]

11 - 位置和输入表示

对应原稿第 13 页

词元（**Token**）身份只是其中之一

除了词元（Token）身份之外，Transformer 还需要某种方式来表示序列顺序。原始 Transformer 在第一层之前将位置编码添加到词元嵌入向量（Embeddings）中。 BERT 使用学习的词元、分段和位置嵌入向量。其他现代模型（Models）在注意力机制（Attention）计算中使用 RoPE，而不是向输入添加位置向量（Vector）。 [4,7]

因此，一般输入表示（Representation）不应被硬编码为

词元嵌入（Token Embedding）+位置嵌入（Embedding）

对于每一个架构。

持久的区别是：

• token embedding：表示离散词汇项；

• 位置机制：为网络提供有关顺序或相对位置的信息；

• 情境化：通过网络混合序列中的信息。

在原始 Transformer 中，在添加位置编码之前，词元（Token）嵌入向量（Embeddings）会乘以 $\sqrt{d_{\text{model}}}$。这种缩放（Scaling）是特定于体系结构的设计选择，而不是嵌入向量查找的通用定义的一部分。 [4]

重要。不要说 RoPE“改变了词元嵌入（Token Embedding）”。标准RoPE作用于Q/K内注意力机制（Attention）；它是围绕上下文计算的位置机制，而不是替代词汇表。

研究依据：[4]、[7]

12 - 输出嵌入向量（Embeddings）和权重绑定（Weight Tying）

对应原稿第 14 页

词汇矩阵（Matrix）可以出现在模型（Model）的两侧

语言模型（Model）最终必须将隐藏状态（Hidden State）转换为词汇项上的逻辑。通过行向量（Vector）约定，可以编写未绑定的输出投影（Output Projection）

$$\ell = hW_{\text{out}}, \qquad W_{\text{out}} \in \mathbb{R}^{d \times V}.$$

Press 和 Wolf 将输出矩阵（Matrix）本身分析为词嵌入（Word Embedding），并建议将输入和输出嵌入向量（Embeddings）绑定到语言模型（Models）中。 [5]

如果输入表是 $E \in \mathbb{R}^{V \times d}$并且权重是绑定的，则logits可以表示为

$$\ell = hE^{\top}$$

（取决于特定于架构的偏差或缩放（Scaling））。

权重绑定（Weight Tying）减少了参数数量，因为模型（Model）不再存储两个独立的

具有相同词汇宽度形状的矩阵。它还耦合了他们的学习：相同的参数行参与表示输入符号和评分输出符号。

Transformer 论文共享了目标输入嵌入向量（Embeddings）和 pre-softmax 线性变换权重，并在其翻译设置中另外共享了源/目标嵌入向量。 [4]

重要。“嵌入矩阵（Embedding Matrix）”可以指输入查找表、被视为嵌入（Embedding）的输出词汇投影，或推理服务（Serving）于这两个角色的绑定矩阵（Matrix）。说明你指的是哪一个。

研究依据：[4]、[5]

13 - 从词元（Tokens）到一个文本向量（Vector）

对应原稿第 15 页

输入可变长度序列，输出固定大小表示（Representation）

检索（Retrieval）和相似性系统通常需要每个句子、段落或文档一个向量（Vector）。然而，Transformer 自然会为每个 词元（Token） 生成一个隐藏状态（Hidden State）：

$$H = [h_1; \ldots; h_T] \in \mathbb{R}^{T \times d}.$$

因此，文本嵌入（Embedding）模型（Model）需要读出规则

$z = g(H, \text{掩模}) \in \mathbb{R}^{d_e}.$

常见的选择包括有效词元（Tokens）的均值池、指定的分类词元、最终的非填充词元、学习的池头或特定于模型（Model）的投影。

Sentence-BERT 显式地向 BERT/RoBERTa 输出添加了池化（Pooling）操作，并研究了平均值、最大值和 CLS 池化。它在原始论文中的默认设置是均值池化。 [9]

现代基于解码器（Decoder）的嵌入（Embedding）模型（Models）可以使用不同的读数。 Qwen3 Embedding 附加一个 EOS 词元（Token），并将 EOS 的最后一层隐藏状态（Hidden State）作为文本表示（Representation）。 [18]

不存在通用的“正确的汇集”。读数是训练模型（Model）定义的一部分。

理解框架（Mental Model）。句子嵌入（Sentence Embedding）不仅仅是“BERT 向量（Vector）”或“LLM 向量”。它是编码器（Encoder）的输出加上特定的读出规则，通常针对预期的相似性任务进行训练或验证。

研究依据：[9]、[14]、[18]、[20]

14 - 池化（Pooling）

对应原稿第 16 页

遮盖填充并保留模型（Model）的预期读数

对于具有隐藏状态 h_i和二进制掩码 $m_i \in \{0,1\}$,的均值池

$$\frac{\sum_{i=1}^{T} m_i h_i}{\sum_{i=1}^{T} m_i}.$$

$$z =$$

掩码（Mask）很重要，因为填充词元（Tokens）不应该默默地将向量（Vector）拉向填充表示（Representation）。

考虑三种有效的词元（Token）状态和一个填充位置：

$$h_1 = [1,0],\ h_2 = [0,2],\ h_3 = [2,1],\ h_4 = [9,9], \quad m = [1,1,1,0].$$

掩蔽均值池给出

$$z = \frac{[1,0] + [0,2] + [2,1]}{3} = [1,1].$$

3

包含填充会产生$[3,3]$,一个完全不同的向量（Vector）。

池化（Pooling）也可以是特定于模型（Model）的。 Sentence Transformers 目前支持的策略包括 CLS、最大值、平均值、加权平均值和最后词元（Token）池。 [20]

重要。池化（Pooling）并不是一种无害的后处理选择。如果模型（Model）是使用特定读数进行训练（Training）的，则更改它可能会使您进入训练目标从未优化过的表示空间（Representation Space）。

研究依据：[9]、[14]、[18]、[20]

15 - 原始 LM 状态并不是自动良好的语义嵌入向量（Embeddings）

对应原稿第 17 页

生成（Generation）目标和相似目标不同

预训练的语言模型（Model）可以为任何文本生成隐藏状态，但这并不能保证简单的池向量（Vector）具有用于语义相似性的有用余弦几何。

Sentence-BERT 在其 2019 年论文中清楚地展示了 BERT 模型（Models）和评估设置：与专门针对句子嵌入向量（Embeddings）进行微调的编码器（Encoder）相比，直接平均 BERT 输出或使用 CLS 输出在语义文本相似性方面表现不佳。重点不是旧的基准数字。关键是客观不匹配。 [9]

生成或掩码语言模型（Model）目标训练隐藏状态以支持词元（Token）预测和模型的内部计算。相反，检索（Retrieval）嵌入（Embedding）目标关心成对几何：在选定的相似性规则下，相关文本应该比不相关文本得分更高。

因此，现代基于 LLM 的嵌入（Embedding）模型（Models）对嵌入任务的基础模型进行了微调，而不是假设基础模型的隐藏状态已经形成了理想的检索（Retrieval）空间。例如，Qwen3 Embedding 通过对比目标和定义的读数来训练基础模型主干。 [18]

重要。“获取最后一个隐藏状态（Hidden State）并对其进行平均”是一个基线（Baseline），而不是高质量嵌入向量（Embeddings）的通用方法。

研究依据：[9]、[11]、[18]、[21]

16 - 连体和双编码器（Encoder）模型（Models）

对应原稿第 18 页

独立编码项目，以低廉的成本进行比较

双编码器（Encoder）将两个输入独立地映射为向量：

$$z_q = f_q(q), \qquad z_d = f_d(d),$$

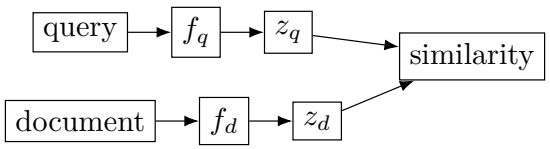
然后计算可分解的相似度，例如

$$s(q, d) = z_q^\top z_d.$$

编码器可以共享参数或者是独立的。 Sentence-BERT 使用连体/三元组风格的结构来使独立的句子表示具有可比性。 DPR 使用不同的问题和段落编码器，两者都基于原始作品中的 BERT。 [9,10]

关键的系统属性是可分解性。可以在不知道未来查询（Query）的情况下预先计算文档向量（Vector）。在查询时，只有查询需要在向量搜索之前进行新的前向传播（Forward Pass）。

交叉编码器（Encoder）则不同：它联合处理该对，这可以模拟更丰富的词元（Token）级交互，但会阻止对每个可能的查询（Query）的文档分数进行独立的预计算。



文件

查询（Query）

相似性

理解框架（Mental Model）。嵌入（Embedding）模型（Models）用一些成对交互的丰富性来换取可重用的表示和可扩展的比较。

研究依据：[9]、[10]、[14]

17 - 相似性：点积（Dot Product）、余弦和 L2

对应原稿第 19 页

该指标是模型（Model）合约的一部分

三种常见的比较是

$$\mathrm{dot}(u, v) = u^\top v,$$

$$\frac{u^\top v}{\|u\|_2 \|v\|_2},$$

$$\cos(u, v) =$$

$$d_2(u, v) = \|u - v\|_2.$$

点积（Dot Product）取决于方向和矢量大小。余弦去除幅度并比较角度。欧几里得距离测量原始尺度中的几何间隔。

如果向量经过 L2 归一化，则 $\|u\| = \|v\| = 1$,则

$$u^\top v = \cos(u, v)$$

和

$$\|u - v\|_2^2 = 2 - 2u^\top v.$$

因此，对于单位向量，最大化余弦/点积（Dot Product）和最小化欧几里德距离会产生相同的排名。

DPR 故意使用点积（Dot Product）。 SimCSE 的标准目标使用余弦相似度（Cosine Similarity）。 E5 使用按温度缩放的余弦。这些是训练（Training）选择，而不仅仅是可互换的查询（Query）时间开关。[10,11,14]

重要。最安全的产生规则是使用嵌入（Embedding）模型（Model）指定的相似函数和归一化（Normalization）约定。即使向量（Vector）维度不变，更改它们也会改变排名。

研究依据：[10]、[11]、[14]、[20]

18 - 标准化

对应原稿第 20 页

单位长度改变了乐谱可以编码的内容

L2 归一化（Normalization）图

$$z \mapsto \hat{z} = \frac{z}{\|z\|_2}.$$

归一化（Normalization）后，每个非零向量（Vector）都位于单位超球面上。大小不再有助于点积相似性。

当训练（Training）目标是有角度的时，这可能是理想的。如果模型（Model）被训练为使用向量（Vector）范数作为分数的一部分，也可能是错误的。

Sentence Transformers 明确指出，归一化（Normalization）嵌入向量（Embeddings）上的点积（Dot Product）相当于余弦相似度（Cosine Similarity），同时避免了比较时的冗余重新归一化。 [20]

归一化（Normalization）还与维度截断相互作用。俄罗斯套娃表示（Representation）学习指出，当使用归一化表示时，应针对每个嵌套维度独立处理归一化，而不是截断归一化全向量（Vector）并假设前缀保持单位长度。 [15]

因此，一个实际的不变量是：

编码约定+标准化约定+度量约定必须一致。

重要。除非模型（Model）明确定义了不对称评分规则，否则不要仅标准化查询（Queries）或仅标准化文档。

研究依据：[11]、[15]、[20]

19 - 工作几何形状示例

对应原稿第 21 页

方向相同，大小不同

采取

$$q = [3, 4], \qquad d_1 = [6, 8], \qquad d_2 = [4, 3].$$

他们的规范是

$$\|q\| = 5, \quad \|d_1\| = 10, \quad \|d_2\| = 5.$$

点积是

$$q^\top d_1 = 50, \qquad q^\top d_2 = 24.$$

余弦相似度为

50

$$\cos(q, d_1) = \frac{50}{5 \cdot 10} = 1,$$

$$\cos(q, d_2) = \frac{24}{5 \cdot 5} = 0.96.$$

欧几里得距离是

$$\|q - d_1\| = 5, \qquad \|q - d_2\| = \sqrt{2} \approx 1.414.$$

所以 d_1与 q完全对齐，但在原始欧几里得空间中更远，因为它的大小加倍了。点积（Dot Product）强烈支持它，因为量级也有贡献。

对所有向量进行归一化后，余弦和点积（Dot Product）一致，欧几里德距离变成相同角度的单调变换。

理解框架（Mental Model）。在指定表示（Representation）缩放（Scaling）和度量之前，“最近”是未定义的。

研究依据：[10]、[20]

20 - 对比学习（Contrastive Learning）

对应原稿第 22 页

用积极和消极的方式塑造空间

现代文本嵌入（Embedding）模型（Models）通常以对比目标进行训练。直觉很简单：增加相关对的相似性，并减少相对于竞争负片的相似性。

锚点 x_i、正值 x_i^+的常见批内目标

$_i$、批大小 N、相似度 s和温度

$$\tau \text{ is}$$

$$\frac{\exp(s(z_i, z_i^+)/\tau)}{\sum_{j=1}^{N} \exp(s(z_i, z_j^+)/\tau)}.$$

$$\ell_i = -\log$$

SimCSE 使用这种具有余弦相似性和批内负数的形式。它的无监督版本使用独立的 dropout mask 生成同一个句子的两种表示；它的监督版本使用蕴涵对作为正例，使用矛盾示例作为硬负例。 [11]

DPR 在正向通道和负向通道上使用类似的 softmax 排名损失。 E5 类似地通过对比学习（Contrastive Learning）和批量负例来训练文本对。 [10,14]

温度控制分数差异对 softmax 的影响程度。较小的 τ会放大相似性的差异，但有用的价值取决于模型和训练（Training）。

重要。对比学习（Contrastive Learning）并不教授“语义相似性”的抽象普遍概念。它根据训练（Training）数据中的正/负对、任务、采样策略和相似函数来教授几何。

研究依据：[10]、[11]、[13]、[14]、[18]

21 - 工作对比损失

对应原稿第 23 页

一种阳性与两种阴性竞争

假设锚点具有余弦相似度（Cosine Similarity）

$$s^+ = 0.8$$

具有积极意义和相似之处

$$s_1^- = 0.3, \qquad s_2^- = 0.0$$

有两个否定。令 $\tau = 0.5$.

指数分数是

$$e^{0.8/0.5} \approx 4.9530, \quad e^{0.3/0.5} \approx 1.8221, \quad e^{0/0.5} = 1.$$

分配给正值的 softmax 概率为

$$p^+ = \frac{4.9530}{4.9530 + 1.8221 + 1} \approx 0.6370.$$

损失是

$$\ell = -\log p^+ \approx 0.4509.$$

如果正相似度上升而负相似度保持不变，则 p^+增加并且损失下降。如果硬负例变得与锚点更加相似，它会收到更多的概率质量，并且损失会增加。

这个玩具计算说明了为什么消极选择很重要。梯度（Gradient）不仅由正对决定，还由分母中存在的替代项决定。

重要。绝对相似度分数在任意嵌入（Embedding）模型（Models）之间不能直接比较。温度、归一化（Normalization）、模型训练（Training）和分数分布都很重要。

与批内底片的连接

在批处理设置中，这个三项分母与下一页上的 B x B 查询（Query）文档得分矩阵的一行的想法相同：一个指定的正项与用作负项的其他批次项竞争。

研究依据：[10]、[11]、[18]

22 - 负片和硬负片

对应原稿第 24 页

分母教导边界

在检索（Retrieval）训练（Training）中，大多数可能的文档与给定的查询（Query）无关。面临的挑战是选择能够教导有用的区别而不引入太多假阴性的否定。

DPR 比较了随机阴性、BM25 检索的阴性以及与其他问题配对的段落。它还展示了批内否定如何重用批次中的其他段落，从 B查询（Query）文档对生成$B \times B$分数矩阵（Score Matrix）。 [10]

对于

$$Q \in \mathbb{R}^{B \times d}, \qquad D \in \mathbb{R}^{B \times d},$$

矩阵（Matrix）

$$S = QD^{\top}$$

包含批次中的每个查询（Query）文档点积（Dot Product）。对角线对可以是正数，而非对角线对如果确实不相关，则可以作为负数。

硬底片特别有价值，因为它们在较弱的检索器（Retriever）下看起来似乎合理，或者与查询（Query）共享表面特征。但实际相关的“负面”会造成矛盾的监督。

现代目标越来越多地包括假阴性过滤或更丰富的阴性池。例如，Qwen3 Embedding 会在基于相似性的标准表明某些候选者的行为可能类似于假阴性时屏蔽某些候选者。 [18]

理解框架（Mental Model）。对比训练（Training）的质量不仅取决于正面例子的结构，还取决于负面例子的结构。

研究依据：[10]、[11]、[14]、[18]

23 - 稠密检索（Retrieval）

对应原稿第 25 页

预先计算文档、对查询（Query）进行编码、搜索向量（Vector）空间

稠密检索器（Retriever）为每个文档或段落构建嵌入（Embedding），将这些向量存储在索引（Index）中，然后嵌入查询（Query）并搜索高分向量。

DPR 形式化了经典的双编码器（Encoder）模式：

$$E_Q(q) \in \mathbb{R}^d, \qquad E_P(p) \in \mathbb{R}^d,$$

$$\mathrm{sim}(q,p) = E_Q(q)^\top E_P(p).$$

昂贵的段落编码器（Encoder）为语料库离线运行。在线查询（Query）成本是一种查询编码加上向量（Vector）搜索。 [10]

这就是为什么嵌入向量（Embeddings）质量和搜索系统质量是不同的问题。对不良嵌入向量的完美精确最近邻搜索仍然会根据错误的几何形状返回邻居。相反，错误的相似性度量、过时的向量、错误的分块（Chunking）或过于激进的近似搜索可能会破坏强大的嵌入向量模型（Model）。

稠密检索（Retrieval）也不会使词汇检索过时。 E5 的分析明确指出了 BM25 保留优势的情况，特别是在精确词汇匹配和某些长尾或长文档任务方面。 [14]

重要。嵌入（Embedding）模型（Model）定义了评分空间。向量（Vector）索引（Index）是一种用于搜索该空间的算法。不要将表示（Representation）学习与索引结构混淆。

研究依据：[10]、[14]

24 - 查询（Query）/文档不对称和说明

对应原稿第 26 页

两侧并不总是意味着同一件事

检索（Retrieval）查询（Query）和文档扮演不同的语义角色。许多嵌入（Embedding）模型（Models）使这种不对称性变得明确。

E5 使用一个共享编码器（Encoder），但在前面添加不同的文本标识符：“query:”用于查询（Queries），“passage:”用于段落。该论文报告说，以这种方式打破对称性对于

一些检索（Retrieval）任务。 [14]

较新的指令感知嵌入（Embedding）模型（Models）概括了这个想法。 Qwen3 Embedding 将任务指令与查询（Query）连接起来，同时保持文档不变，从而允许以请求的任务为条件确定相关性。 [18]

这意味着生产管道必须保留模型（Model）的输入协议。使用查询（Query）模板对语料库进行编码、省略所需的前缀或在索引后更改指令可能会导致不匹配，即使每个向量（Vector）都具有预期的维度。

对于对称任务（例如通用句子相似性），双方可以使用相同的文本角色。对于非对称检索（Retrieval），查询（Query）/文档约定可以是学习几何的一部分。

理解框架（Mental Model）。您嵌入的字符串并不总是用户的原始文本。任务说明、前缀、提示和文档格式可以成为示范合同的一部分。

研究依据：[14]、[18]

25 - 嵌入（Embedding）尺寸

对应原稿第 27 页

容量、存储、带宽和搜索成本

维度 d的嵌入（Embedding）存储 d数字。保持数据类型固定，矢量有效载荷随 d.线性增长

对于存储为 float32 的一百万个向量：

$$1{,}000{,}000 \times 1024 \times 4 \text{ bytes} \approx 3.81 \text{ GiB},$$

而 256 维需要大约

$$0.95 \text{ GiB}.$$

这意味着原始矢量有效载荷减少了4×。真实索引添加元数据、图形结构、量化（Quantization）状态、对齐和其他开销。

更高的维度可以赋予模型（Model）更多的表征能力，但“维度越多越好”是错误的。检索（Retrieval）质量、训练（Training）、数据、架构和索引（Index）行为都与维度相互作用。

危险的捷径是在训练（Training）后截断任意嵌入（Embedding）并假设其第一个坐标是良好的低维表示（Representation）。标准培训并不能保证信息按这种方式排序。

一些现代模型（Models）经过明确的灵活维度训练。 Qwen3 Embedding 报告了 Matryoshka 支持，其官方模型卡在特定于模型的范围内公开了用户可选择的输出尺寸。 [18]

重要。仅当模型（Model）或经过验证的后处理方法支持降维时，降维才是安全的。匹配数组形状并不能证明语义被保留。

研究依据：[15]、[18]

26 - 俄罗斯套娃代表

对应原稿第 28 页

训练有用的前缀而不是盲目截断

俄罗斯套娃表示（Representation）学习 (MRL) 训练高维向量（Vector），以便选定的前缀本身就是有用的表示。如果

$$z \in \mathbb{R}^d,$$

然后对于所选尺寸 $m \in \mathcal{M}$,前缀

$$z_{1:m} \in \mathbb{R}^m$$

针对该任务进行了明确的优化。 [15]

原始 MRL 论文使用嵌套维度，通常以大致对数方式间隔，因此一个模型（Model）可以支持多个精度/效率操作点，而无需重新计算完全不同的表示（Representation）形式。

64 128 256 完整 d

这与 PCA 或训练（Training）后随机获取坐标子集有本质上的不同。 MRL 改变了训练目标，因此有用的信息被打包到嵌套的前缀中。

如果每个前缀都与余弦相似度（Cosine Similarity）一起使用，请在评分之前独立标准化该前缀，而不是重复使用完整向量（Vector）的范数。 [15]

理解框架（Mental Model）。灵活维度是一种学习属性，而不是密集向量的通用属性。

研究依据：[15]、[18]

27 - 各向异性（Anisotropy）和表示（Representation）几何

对应原稿第 29 页

向量（Vector）空间可以折叠成一个狭窄的区域

不保证有用的嵌入空间（Embedding Space）均匀地填充 $\mathbb{R}^d$。上下文和句子表示可以变得各向异性：许多向量占据一个狭窄的圆锥体或由一小组方向主导。 [11,21]

这很重要，因为相似性具有压缩的动态范围（Dynamic Range）。如果几乎每一对都具有很高的余弦相似度（Cosine Similarity），那么将真正相关的示例与不相关的示例区分开来就变得更加困难。

SimCSE 将对比学习（Contrastive Learning）与改进的表示空间（Representation Space）一致性联系起来。其分析认为，对比目标可以使奇异值谱变平，并使句子表示更加各向同性，同时保持正对的对齐。 [11]

各向异性（Anisotropy）并不是一种通用的标量缺陷，只有一种通用的解决方法。后处理方法、归一化（Normalization）、模型（Model）架构、预训练（Pretraining）目标和嵌入（Embedding）微调（Fine-tuning）都可以改变几何形状。

两个有用的概念是：

• 对齐：正值应该接近；

• 均匀性：不相关的样本不应全部塌陷到同一区域。

重要。高维向量（Vector）的有效几何多样性仍然较低。仅凭维度计数并不能告诉您空间是如何使用的。

研究依据：[11]、[21]

28 - 单向量（Vector）、多向量和稀疏表示

对应原稿第 30 页

“嵌入（Embedding）”并不总是意味着一个密集向量

主要的理解框架（Mental Model）是每个文本一个密集向量，但检索（Retrieval）表示更广泛。

单向量（Vector）密集。句子或文档被压缩为一个向量。这是经典的 SBERT/DPR/E5 风格的界面，易于索引（Index）。 [9,10,14]

多向量（Vector）/后期交互。文档可以保留多个标记或区域级向量，并推迟一些交互直到评分。这可以以更高的存储和评分成本保留更细粒度的信息。

稀疏学习表示。模型（Model）可以生成稀疏加权的类似词汇的向量（Vector），而不是密集的低维向量。

BGE-M3 是一个有用的现代示例，因为一个模型（Model）支持稠密检索（Retrieval）、稀疏检索和多向量（Vector）检索。 [17]

这些表示解决了不同的压缩/交互权衡。单个稠密向量（Dense Vector）可以最大程度地方便预计算，但它必须将所有相关信息压缩为一个点。多向量（Vector）方法保留更多局部细节。稀疏表示以学习的形式保留词汇维度。

重要。在讨论现代检索（Retrieval）系统时，不要将“嵌入（Embedding）模型（Model）”用作“单密集向量模型”的同义词。

说明性表示（Representation）尺寸

1,024 维单向缩放因子示每个文档存储 1,024 个标量值。保留 128 个维度为 128 的词元（Token）向量（Vector）可在压缩前存储 16,384 个值 - 数量是原来的 16 倍。真实的多向量系统可以量化、修剪或以其他方式压缩这些表示（Representation），因此这是表示大小的示例，而不是通用存储比率。

研究依据：[9]、[10]、[14]、[17]

29 - 多语言和多模式嵌入向量（Embeddings）

对应原稿第 31 页

共享空间可以跨越语言和模式

相同的表示（Representation）学习思想可以对齐不是用同一种语言编写的对象，或者根本不是文本。

多语言嵌入（Embedding）模型（Models）可以训练（Training）来自不同语言的语义对应文本以占据一个空间的兼容区域。 BGE-M3 和 Qwen3 Embedding 等现代模型报告了跨多种语言的多语言训练和评估。[17,18]

多模式模型（Model）可以对齐不同的数据类型。 CLIP 在图像-文本对上联合训练图像编码器（Encoder）和文本编码器，以便匹配的图像和描述接收兼容的表示。这使得文本到图像和图像到文本检索（Retrieval）成为可能，而无需强制两种模式通过相同的原始输入编码器。 [12]

核心模式依然是

$f_a(x_a)$，$f_b(x_b) \rightarrow$共享比较空间，

但每种模态都可以有自己的编码器（Encoder）和预处理。

共享空间的存在并不意味着向量对于每个任务都是可以互换的。训练（Training）对和目标决定了强调哪些跨模式或跨语言关系。

理解框架（Mental Model）。嵌入（Embedding）是一种表示（Representation）接口。如果训练（Training）将两个领域对齐成兼容的几何形状，那么相似性就可以弥合语言、模式或数据类型。

研究依据：[12]、[17]、[18]

30 - 评估嵌入向量（Embeddings）

对应原稿第 32 页

一个基准分数无法定义通用嵌入（Embedding）

嵌入（Embedding）模型（Models）用于不同的任务：语义文本相似性、检索（Retrieval）、分类、聚类、对分类、重排序（Reranking）支持、双文本挖掘等。

创建 MTEB 的部分原因是仅评估一项任务无法获得完整的图像。其最初的基准涵盖八个任务类别，发现没有单一方法主导该评估中的每项任务。 [16]

不同的任务需要不同的指标：

• STS 通常使用预测相似性和人类评分之间的相关性；

• 检索（Retrieval）使用排名指标，例如召回率（Recall）、MRR、nDCG 或特定于任务的变体；

• 分类评估基于嵌入向量（Embeddings）的分类器或探针；

• 聚类衡量几何结构是否支持有意义的分组。

即使原始余弦分数作为概率校准不佳，检索（Retrieval）模型（Model）也可能非常出色。很好地聚类主题的模型可能不是最好的非对称查询（Query）文档检索器（Retriever）。

基准还取决于语言、领域、文本长度、说明、数据污染和评估实施。目前的 MTEB 项目已经远远超出了原始文本基准，包括更广泛的多语言和多模式评估，因此排行榜声明应始终是日期和基准范围的。 [8]

重要。使用您真正关心的任务和数据分布（而不是全局平均分数）选择一种嵌入（Embedding）模型（Model）。

研究依据：[8]、[16]

31 - RAG 中的嵌入向量（Embeddings）

对应原稿第 33 页

嵌入向量（**Embeddings**）选择候选者；他们不产生答案

在典型的密集 RAG 管道中：

1. 拆分或以其他方式定义可检索单位；

2. 将这些单元编码到文档嵌入向量（Embeddings）中；

3.构建向量（Vector）索引（Index）；

4. 使用正确的查询（Query）协议对用户的查询进行编码；

5.检索高分候选人；

6. 可选择对它们重排序；

7. 将选定的文本发送到生成器（Generator）。

因此，嵌入（Embedding）模型（Model）会影响哪些证据（Evidence）到达 LLM，但它不是生成器（Generator），并且它不包含每个查询（Query）向量（Vector）内的完整外部语料库。

文档嵌入（Document Embedding）是针对相似性目标优化的有损表示（Representation）。它对于候选选择非常有用，因为比较向量比在每个文档上联合运行强大的交叉编码器（Encoder）或生成模型（Model）更便宜。

这会产生几种单独的故障模式：错误的块边界、不理解域的嵌入（Embedding）模型（Model）、查询（Query）/文档格式不匹配、近似搜索未命中或重排序器（Reranker）/生成器（Generator）故障。

重要。 RAG 不是“在嵌入向量（Embeddings）中存储知识”。源文本仍然是证据（Evidence）。嵌入向量是一种用于查找候选文本的检索（Retrieval）表示（Representation）。

研究依据：[10]、[14]、[18]

32 - 生产耦合和版本控制

对应原稿第 34 页

向量仅在兼容的嵌入空间（**Embedding Space**）内具有可比性

存储的向量（Vector）默默地取决于许多选择：

• 型号重量和精确版本；

• 分词器（Tokenizer）和文本预处理；

• 查询（Query）/文档提示或说明；

• 汇集/读出规则；

• 嵌入（Embedding）尺寸或截断设置；

• 标准化；

• 输出数据类型（Dtype）或量化（Quantization）；

• 相似性度量。

更改其中任何一个都可能使新向量与现有索引（Index）不兼容。两个模型（Models）都可以输出 1024 个维度，但位于不相关的坐标系中。

这就是为什么嵌入（Embedding）模型（Model）升级通常需要重新嵌入语料库。尺寸匹配是不够的。即使来自同一系列的新检查点也可以旋转、重新缩放或以其他方式重塑空间。

因此，生产系统应该将嵌入（Embedding）合约与索引（Index）一起版本化。有用的元数据包括模型（Model）标识符、修订版、预处理模板、角色/指令、输出维度、标准化约定和度量。

当前的嵌入（Embedding）库将其中许多公开为配置而不是通用默认值。例如Qwen3，支持用户自定义任务指令，维度灵活； Sentence Transformers 公开了池化（Pooling）和相似性模块。 [18,20]

理解框架（Mental Model）。嵌入（Embedding）索引（Index）与生成它的编码器（Encoder）耦合。将编码器更改视为架构更改，而不是装饰模型（Model）交换。

研究依据：[18]、[20]

33 - 透明的 Pytorch 实现

对应原稿第 35 页

查找、屏蔽均值池和标准化

以下代码显示了三个不同的阶段：词元（Token）查找、池化（Pooling）上下文状态和可选的 L2 标准化。

```
import torch
import torch.nn.functional as F

# 1) Token embedding lookup
V, d = 1000, 64
embedding = torch.nn.Embedding(V, d)
ids = torch.tensor([[4, 1, 7]]) # [B=1, T=3]
x = embedding(ids) # [1, 3, 64]

# 2) Imagine 'hidden' is the contextual Transformer output
hidden = x # pedagogical placeholder
mask = torch.tensor([[1, 1, 1]], dtype=torch.bool)

weights = mask.unsqueeze(-1).to(hidden.dtype)
pooled = (hidden * weights).sum(dim=1) / weights.sum(dim=1)
                                          # [1, 64]

# 3) Unit-normalize if the model contract expects it
z = F.normalize(pooled, p=2, dim=-1)
```

在真实的文本嵌入模型中，隐藏将由上下文编码器产生，并且池化（Pooling）/读出必须与模型（Models）的训练（Training）配置相匹配。有些模型使用均值池，有些模型使用特殊词元（Token），有些模型使用最后/EOS 状态，有些模型使用额外的投影。

重要。该代码是教学性的。不要仅仅因为张量（Tensor）形状有效，就用这个通用的均值池配方替换模型（Model）记录的池化（Pooling）或提示协议。

研究依据：[9]、[18]、[19]、[20]

34 - 常见的实施错误

对应原稿第 36 页

大多数失败是合约不匹配，而不是向量（Vector）算术错误

1. 池化（Pooling）填充词元（Tokens）。平均池通常应该尊重注意力机制（Attention）掩模。

2. 混合标准化和非标准化向量。这默默地改变了分数语义。

3. 使用错误的相似度度量。当范数不同时，余弦、点积（Dot Product）和 L2 的排名可能会有所不同。

4. 省略查询（Query）/文档前缀或指令。一些模型（Models）是使用不对称输入格式进行显式训练的。

5. 在没有 MRL 支持的情况下更改尺寸。任意截断不需要保留有用的几何形状。

6. 比较不同模型（Model）版本的向量。相同的形状并不意味着相同的坐标系。

7. 更改预处理后重用语料库索引（Index）。分词（Tokenization）、模板和截断都可以改变向量。

8. 将原始 LM 隐藏状态视为直接检索（Retrieval）模型（Model）。良好的语言模型表示不会自动针对最近邻语义进行优化。

9. 忘记长输入可能会被截断。所得向量（Vector）可能仅表示保留的前缀或特定于模型（Model）的窗口行为。

理解框架（Mental Model）。写下嵌入（Embedding）合同。模型（Model）、输入格式、读数、维度、标准化和度量应该是显式配置，而不是部落知识。

研究依据：[9]、[14]、[15]、[18]、[20]

35 - 常见的误解

对应原稿第 37 页

创造错误思维模式的捷径

错误观念纠正

“词元（Token） ID 是一种嵌入（Embedding）。”ID 选择一行。向量（Vector）行是嵌入。“每个嵌入维度都有一个含义。”表示通常分布在维度和方向上。“同一个词总是有相同的嵌入。”

它的输入行可能是固定的，但上下文隐藏状态取决于上下文。“BERT/LLM 隐藏状态自动成为良好的检索（Retrieval）嵌入向量（Embeddings）。”

相似几何通常需要合适的读出和训练（Training）目标。“余弦和点积（Dot Product）是相同的。”他们同意单位归一化向量，而不是任意向量。“更多维度总是可以提高检索（Retrieval）效率。”Dimension 将容量与存储/计算进行交换，并且取决于培训。“如果尺寸匹配，您可以比较不同模型（Models）的嵌入向量（Embeddings）。”

不同的编码器定义不同的坐标系。“RAG 将事实存储在向量中。”向量有助于检索源文本；他们不是

源文本本身。

重要。“嵌入”这个词描述的是一种表示，而不是一种通用架构，

目标、合并规则或相似性度量。

研究依据：[4]、[9]、[10]、[15]、[20]

36 - 完整的理解框架（Mental Model）

对应原稿第 38 页

嵌入向量（**Embeddings**）一条链

嵌入向量（Embeddings）是最容易记住的九个想法的序列。

1. 离散化。词元（Token）化将文本映射到词元 ID。

2. 向上看。 ID 从词汇嵌入矩阵（Embedding Matrix）中选择学习行。

3.结合情境。转换器层将基本行转换为上下文相关的隐藏状态。

4. 读出。需要时，池化（Pooling）或特定于模型（Model）的规则将可变长度状态转换为固定大小的文本向量（Vector）。

5.形状几何。对比或针对特定任务的训练（Training）使有用的配对比阴性的得分更高。

6. 选择一个指标。点积（Dot Product）、余弦和 L2 仅在模型（Model）标准化约定下才有意义。

7. 选择容量。嵌入（Embedding）维度控制向量（Vector）有效载荷，并且只有在训练（Training）支持时才能灵活。

8. 索引（Index）。检索（Retrieval）系统预先计算文档向量并对其进行高效搜索；索引与编码器（Encoder）是分开的。

9. 版本合同。模型（Model）、预处理、提示、池化（Pooling）、维度、标准化和指标必须保持兼容。

静态词向量、上下文词元（Token）表示（Representation）、句子嵌入向量（Embeddings）、稠密检索器、俄罗斯套娃向量、多语言空间和多模态空间都是同一个广泛思想的变体：学习一种表示，使下游结构变得更容易计算。

理解框架（Mental Model）。嵌入（Embedding）是为了某种目的而学习的有用几何图形。要理解这一点，请询问映射的内容、矢量是如何产生的、什么目标塑造了空间以及如何比较矢量。

研究依据：[1]–[20]

参考文献

对应原稿第 39 页

主要论文和当前实施来源

1. Y. Bengio、R. Ducharme、P. Vincent、C. Jauvin。神经概率语言模型（Model）。 JMLR 3，2003 年。

2. T. 米科洛夫、K. 陈、G. 科拉多、J. 迪安。向量（Vector）空间中单词表示的有效估计。 arXiv:1301.3781，2013。

3.J.彭宁顿、R.索切尔、C.曼宁。 GloVe：用于单词表示（Representation）的全局向量。 EMNLP 2014。

4.A.瓦斯瓦尼等人。您所需要的就是关注。 NeurIPS 2017.arXiv:1706.03762.

5. O. Press，L. Wolf。使用输出嵌入（Embedding）改进语言模型（Models）。 EACL 2017.arXiv:1608.05859.

6. M.彼得斯等人。深度语境化的单词表示。 NAACL 2018.arXiv:1802.05365.

7. J. Devlin，M.-W。张，K.李，K.图塔诺瓦。 BERT：用于语言理解的深度双向变压器的预训练（Training）。 NAACL 2019.arXiv:1810.04805.

8. MTEB 项目文档和存储库。当前基准测试（Benchmark）范围、任务和评估工具。 2026 年 9 月咨询。

9. N.赖默斯，I.古列维奇。 Sentence-BERT：使用 Siamese BERT 网络的句子嵌入向量（Embeddings）。 EMNLP 2019.arXiv:1908.10084.

10. V. Karpukhin 等人。用于开放域问答的密集段落检索（Retrieval）。 EMNLP 2020.arXiv:2004.04906.

11. 高涛，姚晓，陈东。 SimCSE：句子嵌入向量（Embeddings）的简单对比学习（Contrastive Learning）。 EMNLP 2021.arXiv:2104.08821.

12.A.雷德福德等人。从自然语言监督中学习可迁移的视觉模型（Models）。 ICML 2021.arXiv:2103.00020.

13. A. Neelakantan 等人。通过对比预训练（Training）进行文本和代码嵌入向量（Embeddings）。 arXiv:2201.10005，2022。

14. L. Wang 等人。通过弱监督对比预训练（Training）进行文本嵌入向量（Embeddings）。 arXiv:2212.03533，2022。

15.A.库苏帕蒂等人。俄罗斯套娃表示（Representation）学习。 NeurIPS 2022.arXiv:2205.13147.

16. N. Muennighoff 等人。 MTEB：海量文本嵌入（Embedding）基准。 EACL 2023.arXiv:2210.07316.

17. J. Chen 等人。 BGE M3-Embedding：通过自我知识蒸馏实现多语言、多功能、多粒度文本嵌入向量（Embeddings）。 arXiv:2402.03216，2024。

18. Y. 张等人。 Qwen3 Embedding：通过基础模型（Models）推进文本嵌入（Embedding）和重排序（Reranking）。 arXiv:2506.05176，2025。

19.PyTorch 文档。 torch.nn.Embedding。当前文档查阅于 2026 年 9 月。

20. 句子转换器文档。池化（Pooling）、相似性函数、标准化和嵌入（Embedding）实用程序。当前文档查阅于 2026 年 9 月。

21. B. Li 等人。关于预训练语言模型（Models）的句子嵌入向量（Embeddings）。 EMNLP 2020.arXiv:2011.05864.